

Base HTML (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Local Community Event Portal</title>
</head>
<body>
  <h1>Community Event Portal</h1>

  <select id="categoryFilter">
    <option value="all">All Categories</option>
    <option value="music">Music</option>
    <option value="workshop">Workshop</option>
    <option value="sports">Sports</option>
  </select>

  <input type="text" id="searchBox" placeholder="Search events...">

  <div id="eventContainer"></div>

  <form id="registerForm">
    <input type="text" id="name" placeholder="Your name">
    <input type="email" id="email" placeholder="Your email">
    <select id="eventSelect"></select>
    <button type="submit">Register</button>
    <p id="formError"></p>
  </form>

  <script src="main.js"></script>
</body>
</html>
```

1. JavaScript Basics & Setup

```
console.log("Welcome to the Community Portal");
```

```
window.onload = function () {  
  alert("Page loaded successfully");  
};
```

2. Syntax, Data Types, and Operators

```
const eventName = "Baking Workshop";  
const eventDate = "2026-07-15";  
let seatsAvailable = 20;
```

```
const eventInfo = `${eventName} is happening on ${eventDate}. Seats left: ${seatsAvailable}`;  
console.log(eventInfo);
```

```
function registerSeat() {  
  if (seatsAvailable > 0) {  
    seatsAvailable--;  
    console.log(`Registered! Seats left: ${seatsAvailable}`);  
  } else {  
    console.log("Sorry, this event is full.");  
  }  
}
```

3. Conditionals, Loops, and Error Handling

```
const events = [  
  { name: "Baking Workshop", date: "2026-07-15", seats: 10 },  
  { name: "Rock Concert", date: "2026-06-01", seats: 0 },  
  { name: "Yoga in the Park", date: "2026-08-02", seats: 5 }  
];
```

```
function showUpcomingEvents(eventList) {
  const today = new Date();

  eventList.forEach(function (evt) {
    const eventDate = new Date(evt.date);

    if (eventDate < today) {
      console.log(`${evt.name} has already happened - skipping`);
      return;
    }

    if (evt.seats <= 0) {
      console.log(`${evt.name} is full - skipping`);
      return;
    }

    console.log(`${evt.name} on ${evt.date} - ${evt.seats} seats left`);
  });
}
```

```
showUpcomingEvents(events);
```

```
function registerUserSafely(evt) {
  try {
    if (!evt) throw new Error("Event not found");
    if (evt.seats <= 0) throw new Error("No seats available");
    evt.seats--;
    console.log(`Registered for ${evt.name}`);
  } catch (err) {
    console.error("Registration failed:", err.message);
  }
}
```

4. Functions, Scope, Closures, Higher-Order Functions

```
function addEvent(eventList, newEvent) {  
  eventList.push(newEvent);  
  return eventList;  
}
```

```
function registerUser(evt, userName) {  
  if (evt.seats > 0) {  
    evt.seats--;  
    return `${userName} registered for ${evt.name}`;  
  }  
  return `${evt.name} is full`;  
}
```

```
function filterEventsByCategory(eventList, category) {  
  return eventList.filter(evt => evt.category === category);  
}
```

// Closure example - tracks registrations per category

```
function createRegistrationTracker() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}
```

```
const trackMusicRegistrations = createRegistrationTracker();  
trackMusicRegistrations();  
trackMusicRegistrations();  
console.log(trackMusicRegistrations()); // 3
```

// Higher-order function - callback used for custom search

```
function searchEvents(eventList, callback) {
```

```
    return eventList.filter(callback);  
}
```

```
const results = searchEvents(events, evt => evt.name.toLowerCase().includes("yoga"));  
console.log(results);
```

5. Objects and Prototypes

```
function Event(name, date, seats, category) {  
    this.name = name;  
    this.date = date;  
    this.seats = seats;  
    this.category = category;  
}
```

```
Event.prototype.checkAvailability = function () {  
    return this.seats > 0 ? "Available" : "Full";  
};
```

```
const bakingEvent = new Event("Baking Workshop", "2026-07-15", 10, "workshop");  
console.log(bakingEvent.checkAvailability());
```

```
for (const [key, value] of Object.entries(bakingEvent)) {  
    console.log(`${key}: ${value}`);  
}
```

6. Arrays and Methods

```
const eventArray = [  
    new Event("Baking Workshop", "2026-07-15", 10, "workshop"),  
    new Event("Jazz Night", "2026-07-20", 30, "music"),  
    new Event("5K Fun Run", "2026-07-25", 50, "sports")  
];
```

```
eventArray.push(new Event("Pottery Class", "2026-08-01", 8, "workshop"));
```

```
const musicEvents = eventArray.filter(evt => evt.category === "music");  
console.log(musicEvents);
```

```
const eventCards = eventArray.map(evt => `${evt.name} - ${evt.date}`);  
console.log(eventCards);
```

7. DOM Manipulation

```
function renderEvents(eventList) {  
  const container = document.querySelector("#eventContainer");  
  container.innerHTML = "";  
  
  eventList.forEach(evt => {  
    const card = document.createElement("div");  
    card.className = "event-card";  
    card.innerHTML = `  
      <h3>${evt.name}</h3>  
      <p>${evt.date} | Seats: ${evt.seats}</p>  
      <button class="register-btn" data-name="${evt.name}">Register</button>  
    `;  
    container.appendChild(card);  
  });  
}
```

```
renderEvents(eventArray);
```

```
function updateSeatDisplay(eventName, newSeatCount) {  
  const cards = document.querySelectorAll(".event-card");  
  cards.forEach(card => {  
    if (card.querySelector("h3").textContent === eventName) {  
      card.querySelector("p").textContent = `Seats: ${newSeatCount}`;  
    }  
  })  
}
```

```
});  
}
```

8. Event Handling

```
document.querySelector("#eventContainer").addEventListener("click", function (e) {  
  if (e.target.classList.contains("register-btn")) {  
    const name = e.target.dataset.name;  
    const evt = eventArray.find(ev => ev.name === name);  
    console.log(registerUser(evt, "Guest User"));  
    renderEvents(eventArray);  
  }  
});  
  
document.querySelector("#categoryFilter").addEventListener("change", function (e) {  
  const category = e.target.value;  
  const filtered = category === "all" ? eventArray : filterEventsByCategory(eventArray, category);  
  renderEvents(filtered);  
});  
  
document.querySelector("#searchBox").addEventListener("keydown", function (e) {  
  const query = e.target.value.toLowerCase();  
  const filtered = eventArray.filter(evt => evt.name.toLowerCase().includes(query));  
  renderEvents(filtered);  
});
```

9. Async JS, Promises, Async/Await

```
function fetchEventsFromAPI() {  
  return fetch("https://mockapi.example.com/events")  
    .then(response => response.json())  
    .then(data => {  
      console.log("Events loaded:", data);  
      return data;  
    });  
}
```

```

    })
    .catch(err => {
      console.error("Failed to load events:", err);
    });
  }

  async function loadEventsAsync() {
    const spinner = document.createElement("p");
    spinner.textContent = "Loading events...";
    document.body.appendChild(spinner);

    try {
      const response = await fetch("https://mockapi.example.com/events");
      const data = await response.json();
      renderEvents(data);
    } catch (err) {
      console.error("Error loading events:", err);
    } finally {
      spinner.remove();
    }
  }
}

```

10. Modern JavaScript Features

```

function createEvent(name, date, seats = 10, category = "general") {
  return { name, date, seats, category };
}

```

```

const newEvent = createEvent("Art Fair", "2026-09-01");
console.log(newEvent);

```

```

const { name, date, seats } = newEvent;
console.log(`${name} on ${date}, ${seats} seats`);

```



```
const clonedEvents = [...eventArray];
const availableEvents = clonedEvents.filter(evt => evt.seats > 0);
console.log(availableEvents);
```

11. Working with Forms

```
document.querySelector("#registerForm").addEventListener("submit", function (e) {
  e.preventDefault();
```

```
  const nameInput = document.querySelector("#name").value.trim();
  const emailInput = document.querySelector("#email").value.trim();
  const selectedEvent = document.querySelector("#eventSelect").value;
  const errorBox = document.querySelector("#formError");
```

```
  errorBox.textContent = "";
```

```
  if (!nameInput || !emailInput) {
    errorBox.textContent = "Name and email are required.";
    return;
  }
```

```
  if (!emailInput.includes("@")) {
    errorBox.textContent = "Please enter a valid email.";
    return;
  }
```

```
  console.log(`Registering ${nameInput} (${emailInput}) for ${selectedEvent}`);
});
```

12. AJAX & Fetch API

```
function submitRegistration(userData) {
  fetch("https://mockapi.example.com/register", {
    method: "POST",
```

```

    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(userData)
  })
  .then(response => {
    if (!response.ok) throw new Error("Registration failed");
    return response.json();
  })
  .then(result => {
    console.log("Success:", result);
  })
  .catch(err => {
    console.error("Error:", err.message);
  });
}

// Simulating a delayed server response
function simulateDelayedResponse(userData) {
  console.log("Sending registration...");
  setTimeout(() => {
    console.log(`Registration for ${userData.name} completed`);
  }, 2000);
}

```

13. Debugging and Testing

Since this is a debugging task rather than code to run, here's how you'd actually approach it:

- Open Chrome Dev Tools (F12) and check the Console tab first for red error messages — a silent failure usually still throws something there.
- Go to the Network tab, submit the form, and click the request to `/register`. Check the Payload tab to confirm the data actually being sent matches what you expect.
- Set a breakpoint inside the submit handler (click the line number in the Sources tab) and step through line by line, watching the Scope panel for variable values.
- Add temporary logs at each step to narrow down where it breaks:

```
document.querySelector("#registerForm").addEventListener("submit", function (e) {
  e.preventDefault();
  console.log("Form submitted");

  const nameInput = document.querySelector("#name").value;
  console.log("Name captured:", nameInput);

  const payload = { name: nameInput };
  console.log("Payload ready:", payload);

  fetch("https://mockapi.example.com/register", {
    method: "POST",
    body: JSON.stringify(payload)
  }).then(res => console.log("Response status:", res.status));
});
```

14. jQuery and JS Frameworks

```
$(document).ready(function () {
  $('#registerBtn').click(function () {
    $(this).closest('.event-card').fadeOut();
    alert('Registration successful');
  });

  $('.event-card').fadeIn();
});
```

Gunashree V
111623102066
RMKCET